

# Kotlin

Backend Development

# Network Basics

- **Computer Networks**: Systems that allow computers to communicate

**Internet Protocol (IP)**: System for identifying computers

**Ports**: Communication channels on a computer (0-65535)

# Connections

- **Connection**: **Communication** pathway between two programs
  - **Like** a phone call between two computers
- **Connection Information**:
  - **Source IP** and port
  - **Destination IP** and port
- **TCP Protocol**: **Reliable**, ordered delivery

# IPv4

- An Internet Protocol address (IP address)
- `**x.x.x.x (Address structure)**`:
- `**127.0.0.1 (Localhost)**`:
  - Always refers to your own computer
  - Useful for testing without network access
  - Also accessible as "localhost"
- `**0.0.0.0**`:
  - Means "all available network interfaces"
  - When a server listens on `0.0.0.0`, it accepts connections from any network interface
  - Used to make servers accessible from other computers
- `**255.255.255.255**`: Broadcast address (sends to all devices on local network)
- `**192.168.x.x` and `10.x.x.x**` Private addresses (only work on local networks)

# Sockets

- **Socket**: Programming tool for network communication
- **What Sockets Do**:
  - Create network connections
  - Send data over the network
  - Receive data from the network
  - **Socket Types in Kotlin**:
    - **Socket**: For TCP connections (reliable)
    - **ServerSocket**: For accepting TCP connections
    - **DatagramSocket**: For UDP communication (simpler, faster)

# Client-Server Model

- **Server**: Program that waits for connections and provides a service
  - Listens on a specific port
  - Handles multiple client connections
  - Example: Web server, game server, chat server
- **Client**: Program that connects to a server to use its service
  - Initiates the connection to the server
  - Often has a user interface
  - Example: Web browser, game client, chat app
- **Communication**: Two-way exchange of data

# Server Socket

```
// Create a server socket on port 8080
val serverSocket = ServerSocket(8080)
println("Server started on port 8080")

// Accept one client connection
val clientSocket = serverSocket.accept()
println("Client connected!")

// Create a writer to send data to the client
val writer = PrintWriter(clientSocket.getOutputStream(), true)

// Send a message
writer.println("Hello from Kotlin server!")

// Clean up
clientSocket.close()
serverSocket.close()
println("Server stopped")
```

# Client Socket

```
// Connect to the server (localhost = 127.0.0.1)
val socket = Socket("localhost", 8080)
println("Connected to server")

// Set up reader to receive data
val reader = BufferedReader(InputStreamReader(socket.getInputStream()))

// Read and print the message from server
val message = reader.readLine()
println("Server says: $message")

// Clean up
socket.close()
println("Connection closed")
```

# What is Backend?

- The server-side of an application that users don't directly interact with
- Handles core business logic, data storage, and security
- Communicates with other systems (databases, services, etc.)
- Serves data and functionality to frontends (web, mobile apps)
- Runs on servers or cloud platforms
- Often built as a set of services or APIs

# Web APIs

- APIs that communicate over the internet using HTTP
- Client-server communication model
- Common formats: JSON, XML
- Technology agnostic (different languages can communicate)
- Types:
  - SOAP: Uses XML, more rigid structure
  - REST: Uses HTTP methods, resource-based
  - GraphQL: Query language for APIs, flexible data fetching
  - RPC: Remote Procedure Call, action-based

# HTTP

- text based communication protocol (http 1.\*)
- can handle text and binary data

# Http Request

```
curl -v http://google.com
```

```
* Host google.com:80 was resolved.
```

```
* IPv6: (none)
```

```
* IPv4: 216.58.206.46
```

```
*   Trying 216.58.206.46:80...
```

```
* Connected to google.com (216.58.206.46) port 80
```

```
> GET / HTTP/1.1
```

```
> Host: google.com
```

```
> User-Agent: curl/8.7.1
```

```
> Accept: */*
```

```
>
```

```
* Request completely sent off
```

# Http Response

< HTTP/1.1 301 Moved Permanently

< Location: <http://www.google.com/>

< Content-Type: text/html; charset=UTF-8

< Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-src 'nonce-z-LCHgth0e1AZCmPgxyuFA' 'strict-dynamic' 'report-sample' 'unsafe-eval' 'unsafe-inline' https: http;;report-uri https://csp.withgoogle.com/csp/gws/other-hp

< Date: Mon, 05 May 2025 10:57:07 GMT

< Expires: Wed, 04 Jun 2025 10:57:07 GMT

< Cache-Control: public, max-age=2592000

< Server: gws

< Content-Length: 219

< X-XSS-Protection: 0

< X-Frame-Options: SAMEORIGIN

# Http Response

<

```
<HTML><HEAD><meta http-equiv="content-type" content="text/html; charset=utf-8">
```

```
<TITLE>301 Moved</TITLE></HEAD><BODY>
```

```
<H1>301 Moved</H1>
```

The document has moved

```
<A HREF="http://www.google.com/">here</A>.
```

```
</BODY></HTML>
```

\* Connection #0 to host google.com left intact

# REST

- REST = Representational State Transfer
- Architectural style for web APIs created by Roy Fielding (2000)
- Based on key principles:
  - Uses HTTP methods (GET, POST, PUT, DELETE)
  - Resource-based (URLs represent resources)
  - Stateless (server doesn't store client state)
  - Uniform interface (consistent interaction patterns)
  - Client-server separation (independent evolution)

# Http Methods in REST

- **\*\*GET\*\***: Retrieve data (read-only, safe)
  - Example: ``GET /users/123`` - Get user with ID 123
- **\*\*POST\*\***: Create new resources
  - Example: ``POST /users`` - Create a new user
- **\*\*PUT\*\***: Update existing resources (full replacement)
  - Example: ``PUT /users/123`` - Update entire user 123
- **\*\*PATCH\*\***: Partially update resources
  - Example: ``PATCH /users/123`` - Update parts of user 123
- **\*\*DELETE\*\***: Remove resources
  - Example: ``DELETE /users/123`` - Delete user 123

# REST Resources and URLs

- Resources are the key objects in your system
  - Example: users, products, tasks
- URLs (endpoints) identify resources
  - Collection: ``/users`` (all users)
  - Individual: ``/users/123`` (specific user)
- Hierarchical relationships
  - ``/users/123/orders`` (orders for user 123)
- Query parameters for filtering, sorting, pagination
  - ``/products?category=electronics&sort=price``

# HTTP Status Codes

- Numeric codes that indicate request outcome
- Grouped by first digit:
  - **\*\*2xx\*\***: Success
    - 200 OK, 201 Created, 204 No Content
  - **\*\*3xx\*\***: Redirection
    - 301 Moved Permanently, 304 Not Modified
  - **\*\*4xx\*\***: Client Error
    - 400 Bad Request, 401 Unauthorized, 404 Not Found
  - **\*\*5xx\*\***: Server Error
    - 500 Internal Server Error, 503 Service Unavailable
- Helps clients understand what happened

# REST API Benefits

- **Scalability**: Stateless nature helps scaling
- **Flexibility**: Clients can evolve independently
- **Portability**: Works across platforms and languages
- **Visibility**: Clear HTTP interactions
- **Reliability**: Well-understood caching and error handling
- **Simplicity**: Easier to understand and implement
- **Widespread adoption**: Industry standard

# Backend Framework Basics

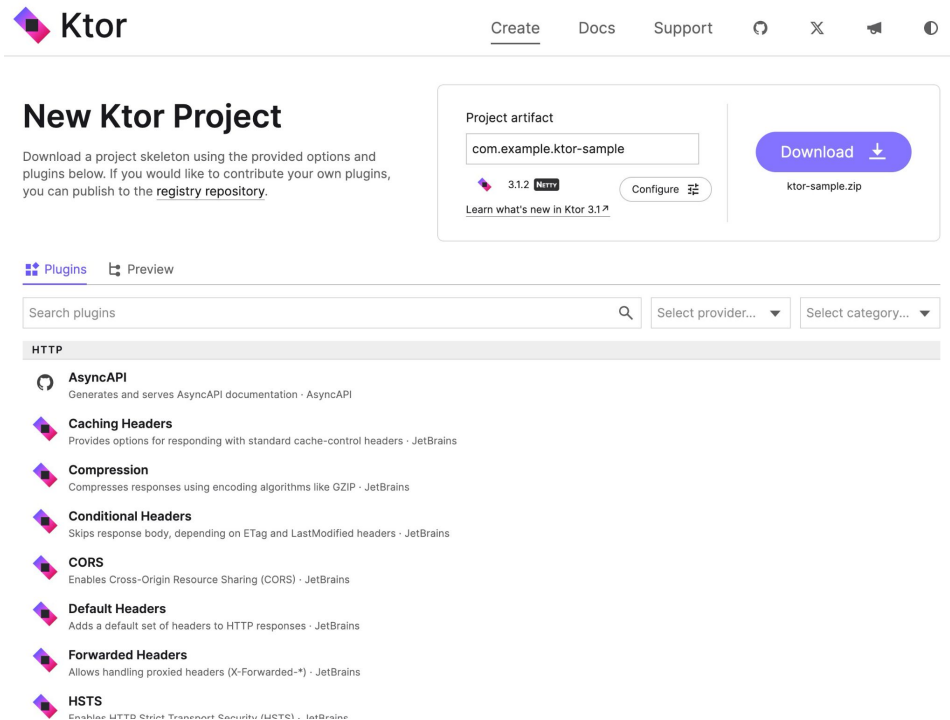
- Frameworks provide structure and common functionality
- Handle repetitive tasks (routing, serialization, etc.)
- Common features:
  - HTTP server functionality
  - Routing system
  - Request/response handling
  - Data validation
  - Database integration
  - Authentication/Authorization
  - Testing support
  - Logging and monitoring

# Introduction to Ktor

- Close to HTTP
- Lightweight, asynchronous web framework for Kotlin
- Developed by JetBrains
- Ideal for microservices, HTTP APIs, and RESTful applications
- Current version: 3.1.2 (March 2025)
- Highly modular with plugin-based architecture

# Setting Up a Ktor Project

- Use the Ktor Project Generator at <https://start.ktor.io>
- Select your preferences:
  - Build system (Gradle with Kotlin DSL recommended)
  - Ktor version
  - Server engine (Netty, CIO, Tomcat, etc.)
  - Required plugins



The screenshot shows the Ktor Project Generator website. At the top, there's a navigation bar with links for 'Create', 'Docs', 'Support', and social media icons. The main heading is 'New Ktor Project'. Below it, a description states: 'Download a project skeleton using the provided options and plugins below. If you would like to contribute your own plugins, you can publish to the [registry repository](#).' The form includes a 'Project artifact' field with the value 'com.example.ktor-sample', a 'Download' button, and a 'Configure' button. A version indicator shows '3.1.2' with a 'New' badge. A link to 'Learn what's new in Ktor 3.1' is also present. Below the form, there's a 'Plugins' section with a search bar and dropdowns for 'Select provider...' and 'Select category...'. A list of HTTP plugins is displayed, including AsyncAPI, Caching Headers, Compression, Conditional Headers, CORS, Default Headers, Forwarded Headers, and HSTS, each with a brief description and the provider 'JetBrains'.

**Ktor**

Create Docs Support

## New Ktor Project

Download a project skeleton using the provided options and plugins below. If you would like to contribute your own plugins, you can publish to the [registry repository](#).

Project artifact  
com.example.ktor-sample

Download 

ktor-sample.zip

3.1.2 **New** Configure 

[Learn what's new in Ktor 3.1](#)

Plugins Preview

Search plugins  Select provider... Select category...

**HTTP**

- AsyncAPI**  
Generates and serves AsyncAPI documentation - AsyncAPI
- Caching Headers**  
Provides options for responding with standard cache-control headers - JetBrains
- Compression**  
Compresses responses using encoding algorithms like GZIP - JetBrains
- Conditional Headers**  
Skips response body, depending on ETag and LastModified headers - JetBrains
- CORS**  
Enables Cross-Origin Resource Sharing (CORS) - JetBrains
- Default Headers**  
Adds a default set of headers to HTTP responses - JetBrains
- Forwarded Headers**  
Allows handling proxied headers (X-Forwarded-\*) - JetBrains
- HSTS**  
Enables HTTP Strict Transport Security (HSTS) - JetBrains

# Starting Ktor application

```
fun main() {  
    // Start Ktor server with Netty engine  
    embeddedServer(  
        Netty,           // Engine  
        port = 8080,     // Port  
        host = "0.0.0.0", // Host  
        module = Application::module // Module function  
    ).start(wait = true)  
}
```

# Simple Ktor Application

```
fun Application.module() {  
    routing {  
        get("/") {  
            call.respondText("Hello World!")  
        }  
    }  
}
```

# Building Routes

```
fun Application.configureRouting() {  
    routing {  
        // Define a GET endpoint  
        get("/") {  
            call.respondText("Hello, Ktor!")  
        }  
  
        // Define a GET endpoint with JSON response  
        get("/info") {  
            call.respond(mapOf("version" to "1.0"))  
        }  
    }  
}
```

# Building Routes

```
route("/api") {  
  // Nested routes  
  route("/v1") {  
    // Tasks API endpoints  
    route("/tasks") {  
      get { /* Get all tasks */ }  
      post { /* Create a task */ }  
  
      // Path parameters  
      route("/{id}") {  
        get { /* Get task by ID */ }  
        put { /* Update task */ }  
        delete { /* Delete task */ }  
      }  
    }  
  }  
}
```

# Request Parameters

```
// /tasks/42
get("/tasks/{id}") {
    val id = call.parameters["id"]?.toIntOrNull()
    if (id == null) {
        call.respond(HttpStatusCode.BadRequest, "Invalid ID format")
        return@get
    }
    // Use the ID parameter
}
```

# Query Parameters

```
// /tasks?id=42
```

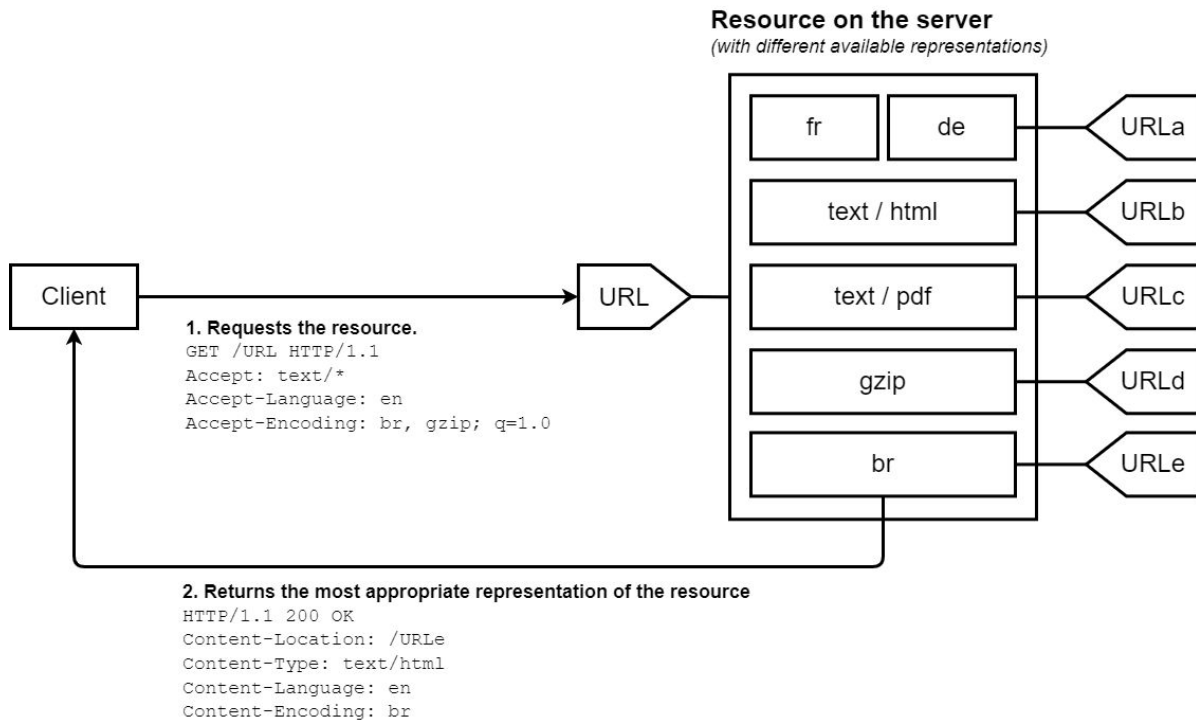
```
get("/tasks") {  
    val id = call.queryParameters["id"]?.toIntOrNull()  
    if (id == null) {  
        call.respond(HttpStatusCode.BadRequest, "Invalid ID format")  
        return@get  
    }  
    // Use the ID parameter  
}
```

# Plugins in Ktor

- ContentNegotiation: Handles serialization/deserialization
- Routing: Manages HTTP route definitions
- Authentication: Handles user authentication
- CORS: Manages cross-origin resource sharing
- StatusPages: Handles errors and exceptions
- Compression: Compresses HTTP responses
- CallLogging: Logs HTTP requests and responses

# Content Negotiation

[https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Content\\_negotiation](https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Content_negotiation)



# Content Negotiation

```
fun Application.configureSerialization() {  
    install(ContentNegotiation) {  
        // Use kotlinx.serialization for JSON  
        json()  
  
        // Alternative: Use Gson  
        // gson()  
  
        // Alternative: Use Jackson  
        // jackson()  
    }  
}
```

# Content Negotiation

```
@Serializable
data class Task(
    val id: Int? = null,
    val title: String,
    val description: String,
    val completed: Boolean = false
)
```

# Content Negotiation: send

```
fun Application.module() {  
    routing {  
        get("/") {  
            call.respond(HttpStatusCode.OK, Task(...))  
        }  
    }  
}
```

# Content Negotiation: receive

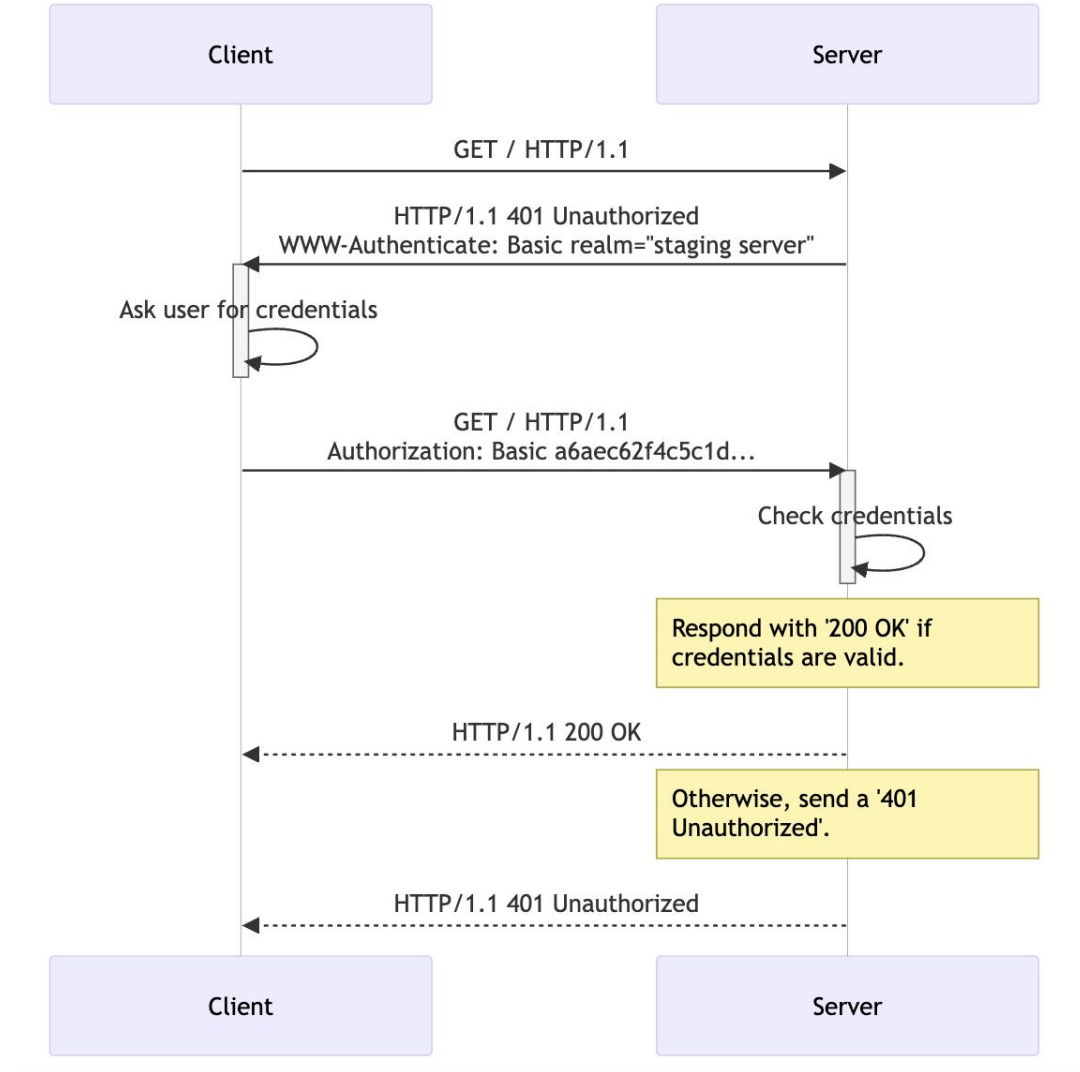
```
post("/tasks") {  
    try {  
        // Parse the request body into a Task object  
        val task = call.receive<Task>()  
  
        // Process the task  
        // ...  
  
        // Respond with the created task  
        call.respond(HttpStatusCode.Created, task)  
    } catch (e: ContentTransformationException) {  
        call.respond(  
            HttpStatusCode.BadRequest,  
            "Invalid task format: ${e.message}"  
        )  
    }  
}
```

# Error Handling

```
fun Application.configureStatusPages() {  
    install(StatusPages) {  
        // Handle exceptions thrown during routing  
        exception<Throwable> { call, cause →  
            call.respond(HttpStatusCode.BadRequest, "Invalid request format")  
        }  
        ...  
        status(HttpStatusCode.NotFound) { call, status →  
            call.respond(  
                status,  
                mapOf("error" to "The requested resource was not found")  
            )  
        }  
    }  
}
```

# Authentication

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Authentication>



# Basic Authentication Config

```
fun Application.configureAuthentication() {  
    install(Authentication) {  
        // Basic authentication  
        basic("auth-basic") {  
            validate { credentials →  
                if (credentials.name == "admin" &&  
                    credentials.password == "password") {  
                    UserIdPrincipal(credentials.name)  
                } else {  
                    null  
                }  
            }  
        }  
    }  
}
```

...

# Authentication Use

```
routing {  
    // Public routes  
    get("/public") {  
        call.respondText("Public endpoint")  
    }  
  
    // Protected routes  
    authenticate("auth-basic") {  
        get("/protected") {  
            val principal = call.principal<UserIdPrincipal>()  
            call.respondText("Protected endpoint, user: ${principal?.name}")  
        }  
    }  
}
```

# Testing

```
class TaskApiTest {  
  
    @Test  
    fun `get all tasks returns 200`() = testApplication {  
        application {  
            configureRoutine()  
        }  
  
        val client = createClient {  
            install(ContentNegotiation) {  
                json()  
            }  
        }  
    }  
    ...  
}
```

# Testing

```
// Perform test request
```

```
val response = client.get("/tasks")
```

```
// Verify response
```

```
assertEquals(HttpStatusCode.OK, response.status)
```

```
val tasks = response.body<List<Task>>()
```

```
assertEquals(2, tasks.size)
```

```
}
```

```
}
```

# Database Integration

- `Ktor` doesn't include built-in database support
- `Common` choices:
  - `**Exposed**`: `Kotlin SQL` library by `JetBrains`
  - `**JDBC**`: `Standard Java` database connectivity
  - `**MongoDB**`: `NoSQL` database with `Kotlin` driver
  - `**Redis**`: `In-memory data` structure store
- `Create` database-backed repository implementations

# Exposed Setup

```
dependencies {  
    implementation("org.jetbrains.exposed:exposed-core:0.61.0")  
    implementation("org.jetbrains.exposed:exposed-dao:0.61.0")  
    implementation("org.jetbrains.exposed:exposed-jdbc:0.61.0")  
    implementation("com.h2database:h2:2.1.214") // H2 database  
}
```

# Define Table

```
object Tasks : Table("tasks") {  
    val id = integer("id").autoIncrement()  
    val title = varchar("name", MAX_VARCHAR_LENGTH)  
    val description = varchar("description", MAX_VARCHAR_LENGTH)  
    val isCompleted = bool("completed").default(false)  
}
```

# Connect and create tables

```
fun Application.configureDatabases() {  
    val database = Database.connect(  
        url = "jdbc:h2:mem:test;DB_CLOSE_DELAY=-1",  
        driver = "org.h2.Driver"  
    )  
  
    transaction(database) {  
        // Create tables  
        SchemaUtils.create(Tasks)  
    }  
}
```

# Inserting Data

```
val taskId = Tasks.insert {  
    it[title] = "Learn Exposed"  
    it[description] = "Go through the Get started with Exposed tutorial"  
} get Tasks.id
```

```
val secondTaskId = Tasks.insert {  
    it[title] = "Read The Hobbit"  
    it[description] = "Read the first two chapters of The Hobbit"  
    it[isCompleted] = true  
} get Tasks.id
```

# Selecting Data

```
Tasks.select(Tasks.id.count(), Tasks.isCompleted).groupBy(Tasks.isCompleted).foreach {  
    println("${it[Tasks.isCompleted]}: ${it[Tasks.id.count()]} ")  
}
```

# Repository Pattern

- **What**: Design pattern to abstract data access logic
- **Why**: Separates business logic from data storage
- **Benefits**:
  - Testability (can mock repositories)
  - Flexibility (easily swap implementations)
  - Maintainability (consistent data access API)
  - Single Responsibility Principle

# Task Repository

```
interface TaskRepository {  
    // Get all tasks  
    suspend fun getAllTasks(): List<Task>  
  
    // Get task by ID  
    suspend fun getTaskById(id: Int): Task?  
  
    // Create a new task  
    suspend fun createTask(task: Task): Task  
    ...  
}
```

# Testing Repository

```
class InMemoryTaskRepository : TaskRepository {  
    private val idCounter = AtomicInteger(1)  
    private val tasks = ConcurrentHashMap<Int, Task>()  
  
    init {  
        // Add some initial tasks  
        createTask(Task(title = "Setup project", ...))  
        createTask(Task(title = "Learn Ktor", ...))  
    }  
  
    override suspend fun getAllTasks(): List<Task> = tasks.values.toList()  
  
    override suspend fun getTaskById(id: Int): Task? = tasks[id]
```

# Running in Production

- Package application as a JAR or WAR
- Deploy to:
  - Cloud providers (AWS, GCP, Azure)
  - Kubernetes
  - Docker containers
  - Java servlet containers (Tomcat, Jetty)
- Configuration via environment variables
- Health checks and monitoring
- Using Netty or CIO engines for best performance

# Health check

```
routing {  
  // Simple health check  
  get("/health") {  
    call.respond(HttpStatusCode.OK, mapOf("status" to "UP"))  
  }  
}
```

# Health check

```
get("/health/detailed") {  
    val databaseStatus = try {  
        // Check database connection  
        taskRepository.ping()  
        "UP"  
    } catch (e: Throwable) {  
        "DOWN"  
    }  
  
    // Determine overall status  
    val overallStatus = if (databaseStatus == "UP") "UP" else "DOWN"  
  
    call.respond(  
        if (overallStatus == "UP") HttpStatusCode.OK else HttpStatusCode.ServiceUnavailable,  
        mapOf(  
            "status" to overallStatus,  
            "components" to mapOf(  
                "database" to databaseStatus  
            ),  
            "version" to "1.0.0"  
        )  
    )  
}
```

# Metrics and Monitoring

```
implementation("io.ktor:ktor-server-metrics-micrometer")
```

```
implementation("io.micrometer:micrometer-registry-prometheus:1.11.0")
```

# Metrics and Monitoring

```
install(MicrometerMetrics) {  
    registry = appMicrometerRegistry  
  
    // Tag all metrics with application name  
    meterBinders = listOf(  
        ClassLoaderMetrics(),  
        JvmMemoryMetrics(),  
        JvmGcMetrics(),  
        ProcessorMetrics(),  
        JvmThreadMetrics()  
    )  
}
```

# Metrics and Monitoring

```
routing {  
    get("/metrics") {  
        call.respond(appMicrometerRegistry.scrape())  
    }  
}
```

# Deployment Options

- **JAR** deployment:
  - `./gradlew shadowJar``
  - `java -jar build/libs/application.jar``
- **Docker**:
  - Simple Dockerfile provided by Ktor
  - `docker build -t myapp .``
  - `docker run -p 8080:8080 myapp``
- **Kubernetes**:
  - Define resource requirements
  - Health checks for liveness/readiness
  - Auto-scaling based on metrics
- **Cloud** platforms

# Resources

- **Official Documentation**: [ktor.io/docs](https://ktor.io/docs)
- **GitHub Repository**: [github.com/ktorio/ktor](https://github.com/ktorio/ktor)
- **Samples**: [github.com/ktorio/ktor-samples](https://github.com/ktorio/ktor-samples)
- **Project Generator**: [start.ktor.io](https://start.ktor.io)